

二分法を用いた故障の支配関係候補の効率的な削減方法

松永 裕介^{1,a)}

概要：故障 f_1 を検出するいかなるテストパターンも故障 f_2 を検出する時、故障 f_1 は故障 f_2 を支配するという。この支配関係を利用することで、テストパターン生成時に考慮すべき故障数を削減することができる。故障の支配関係は SAT ソルバを用いることで厳密に検査することが可能であるが、単純には全故障数の二乗に比例した回数の検査を必要とするため大規模な回路に対しては実用的ではない。一方、ある入力パターンによって故障 f_1 が検出されて故障 f_2 が検出されない場合、SAT ソルバによる検査を行うことなく支配関係がないことがわかる。そこで、故障シミュレーションを用いて検査対象の故障対候補を削減する方法が考えられるが、この故障対候補の削減も単純な手法では全故障数の二乗に比例した手間を必要とする。本稿では故障シミュレーションによって検出される故障グループと検出されない故障グループに二分することで故障間の支配関係を段階的に詳細化していく手法を提案する。ベンチマーク回路を用いた実験の結果、大規模回路に対しても効率よく故障対候補の削減を行えることを確認している。

キーワード：テストパターン生成、故障シミュレーション、充足可能性判定問題

Dichotomy Based Reduction Method for Fault Dominance Candidates

Abstract: Fault f_1 is said to dominate f_2 if any testpattern which detects f_1 always detects also f_2 . This dominance relation can reduce the size of faults set to be considered in testpattern minimization. Though SAT solver exactly and efficiently checks the fault dominance relation, the number of candidates of dominance checking is proportional to the square of the number of faults. Thus, it is not suitable for large scale circuits. On the other hand, if a testpattern detects fault f_1 but does not detect fault f_2 , one can prove that f_1 does not dominate f_2 , so SAT solver does not need to check the dominance relation between f_1 and f_2 . This fault simulation based method drastically reduces the number of candidates to be checked by SAT solver. However, the time complexity of this method is also proportional to the square of the number of faults. This paper proposes a novel efficient method which reduce the number of candidates using the dichotomy technique. Experiments using benchmark circuits show that the proposed method can efficiently reduce the number of candidates.

Keywords: test pattern generation, fault simulation, SAT

1. はじめに

製造した LSI に故障がないかどうかを検査するためテストパターンを生成する問題をテスト (パターン) 生成問題と呼ぶ。テストパターンは仮定された故障をすべて検出できることが要求されるが、さらに LSI1 つあたりの検査時間の短縮や検査コストの低減のため、テストパタンの要素数は少ないことが望ましい。

ある故障を検出するテストパターンが他の故障も検出でき

る場合がある。特殊な場合として故障 f_1 を検出するいかなるテストパターンも必ず他の故障 f_2 を検出できるケースが存在する。このような場合、故障 f_1 を検出するテストパターンを考慮すれば常に故障 f_2 も検出されるので明示的に故障 f_2 の検出を考える必要はない。本稿ではこのような関係を故障の支配関係と呼ぶ。与えられた全ての故障を検出するテストパターン集合を求める問題を考える時、対象の故障を他の故障から支配されていない故障のみに限定しても解は変わらない。テストパターンを最小化するための手法は古くからさまざまなものが提案されているが、対象の回路規模が大きくなるに従って効率よく小さなテストパターン集合を求

¹ 九州大学
744 Motoooka, Nishi-ku, Fukuoka, 819-0395, Japan

^{a)} matsunaga.yusuke.676@m.kyushu-u.ac.jp

めることが難しくなっている。そこで、考慮すべき故障の数を削減することができれば要素数の少ないテストパターン集合を生成する際に有効であると考えられる。本稿では故障の支配関係に基づいて考慮すべき故障の数を削減する手法について述べる。

LSI のテスト生成問題は CNF-SAT 問題に定式化できることは古くから知られており [2], 2000 年代に入って Chaff[6], [7] や GRASP[3], MINISAT などの優れた SAT ソルバが開発されたため、大規模な回路のテスト生成に関しては SAT ソルバを用いたものが効率的であることが知られている。故障 f_1 が故障 f_2 を支配しているかどうかの検査もテスト生成問題と同様に CNF-SAT 問題に定式化することができ、SAT ソルバを用いて解くことができる。この問題は NP 完全問題ではあるが、比較的短い時間で解を求めることが可能である。ところが、故障の支配関係を調べるためには全故障対に対して検査を行う必要があり、全故障対の数は全故障数の二乗となる。この二乗のオーダーは大規模回路に対しては適用不可でありなんらかの効率化が必要である。

あるパターンを入力として故障シミュレーションを行った結果、故障 f_1 は検出され、故障 f_2 は検出されなかったとする。すると、この時点で故障 f_1 は故障 f_2 を支配しないことがわかる。このように故障シミュレーションの結果を用いることで支配関係の検査対象の候補するを削減することが可能である。ところが故障シミュレーションを用いる場合でも、単純な手法では対象の故障対が支配関係の検査の候補かどうか（支配関係の可能性があるか）を調べるために全故障数の二乗のオーダーの計算を必要とするため、根本的な解決にはなっていない。

そこで本稿では故障の代わりに故障集合間の支配関係を調べ、故障集合を故障シミュレーションの結果で分割してゆく手法を提案する。初期状態では対象の故障集合はただ一つであり、支配関係の検査対象も自分自身のみである。この故障集合に対してあるパターンで故障シミュレーションを行うと、検出できた故障の集合と検出できなかった故障の集合に分割される。この時点で検出できた故障の集合は検出できなかった故障の集合を支配する可能性がないことがわかる。分割された個々の故障集合に対して別のパターンの故障シミュレーションを行うことでさらに細分化を行う。このような処理を変化がなくなるまで繰り返すことで、各故障集合は細分化され、多くの場合は単一の故障を持つ集合となる。もしも複数の故障を持つ集合が残った場合にはそれらの故障は等価故障の可能性が高いので等価故障の検査を行う。故障グループ間の支配関係をその要素である故障の支配関係に反映することで支配関係の検査対象の故障対を求めることができる。最終的にそれらの故障対に対して支配関係の検査を行うことで支配故障を求めることができ、最小テストパターン生成問題と考慮すべき故障数を削減

することができる。ベンチマーク回路を用いた実験の結果、提案手法は大規模な回路に対しても効率よく支配故障を求めることに成功している。また、支配故障数は全故障数の 30%~40% であり、支配関係を用いた故障数削減が効果的であることを示している。

以下、2 節でテスト生成問題と SAT ソルバを用いたテスト生成手法の用語および基本概念について説明する。3 節で二分法を用いた故障対候補の削減手法について説明し、4 節でベンチマーク回路を用いた実験結果を示す。

2. テスト生成問題と SAT ソルバを用いたテスト生成手法

本節では、テスト生成問題の定義と SAT ソルバを用いた手法の説明を行なう。本稿では組み合わせ論理回路を対象とする。同期式順序回路の場合には、フリップフロップをスキャンタイプにするなどの適切な処理で、対象を組み合わせ回路とみなせると仮定する。

2.1 テストパターン生成問題

ある故障 f を考える。通常は、故障 f の存在する回路 (以下、 f の故障回路と呼ぶ^{*1}) の振る舞いは故障の存在しない回路 (以下、正常回路と呼ぶ) の振る舞いと異なる。テストパターン生成問題 (テスト生成問題) とは、与えられた回路と故障 f に対して、正常回路と故障回路の出力が異なるような入力を求めることである。そのような入力を故障 f に対する**テストパターン**と呼ぶ。逆にそのような入力は故障 f を**検出する**と言う。場合によっては、ある故障 f' に対する故障回路の振る舞いが正常回路とまったく変わらないことがある。そのような場合には、その故障 f' を検出することはできない。実際、故障 f' の存在は回路の振る舞いに影響を与えていないので、そのような故障は**冗長故障**と呼ばれる。

一般のテスト生成問題とは、対象の回路と複数の故障が与えられ、その故障のうち検出可能な故障に対しては、そのテストパターンを求め、残った故障に対しては、それが冗長故障であると特定 (証明) することである。

本稿では、単一縮退故障を対象とする^{*2}。つまり、ここでいう故障とは、回路中のただ一ヶ所において、その値が常に 0 または 1 に固定 (縮退) してしまうもののことである。

2.2 SAT ソルバを用いた基本的な手法

まず SAT 問題に関する用語の定義を行う。

変数 v : $\text{true}(1) \mid \text{false}(0)$ のどちらかをとり変数。便宜上、値の割り当てられていないことを表す X も含めた 3 値を取るように扱うこともある。

^{*1} 文脈上、どの故障に対するものかが明らかな場合には単に故障回路と呼ぶ

^{*2} スキャン方式を用いた単一遷移故障のテストパターン生成でも同様の議論が成り立つ。

リテラル $l = v$ か $\neg v$: 変数の肯定もしくは否定を表すもの。あるいは1変数論理関数とみなすこともできる。 v を肯定リテラル $\neg v$ を否定リテラルとよぶ。肯定リテラルはもとの変数が true の時 true となる (充足される)。否定リテラルはもとの変数が false の時 true となる (充足される)。

節 $c = (l_1 \vee l_2 \vee \dots \vee l_i \vee \dots)$: リテラルの論理和。どれか一つのリテラルが true となったら節は充足されたと定義する。

CNF(Conjunctive Normal Form) : 節の論理積。全ての節が充足されたとき、CNF 全体が充足されたと定義する。

CNF-SAT 問題 : 与えられた CNF が充足可能かどうか判定する問題。充足可能の場合、充足解 (充足させる変数の割り当て) の一つを出力することが多い。さらに、充足解として、全ての変数に値が割り当てられているもの (完全割り当て解) を出力することが多い。一部の变数の値が未割り当てのものを部分割り当て解と呼ぶこともある。充足不可能の場合はただ単に充足不能と出力する場合と、Davis-Putnam アルゴリズム [1] に基づいたリゾリューション (resolution) を出力する場合がある。割り当ての集合 (リテラルの論理積) α が CNF 式 ψ を充足させる時、 $\alpha \models \psi$ と表記する。 α を ψ のモデル (model) と呼ぶ。

ここでは、テスト生成問題を SAT ソルバを用いて解く基本的な手法について説明する。大まかには図 1 に示すように、元となる回路 (正常回路) と故障 f が存在する回路 (故障回路) を並べたテスト生成用の合成回路を作成する。合成回路の外部入力数は元の回路の外部入力数と同一であり、それぞれが対応する正常回路と故障回路の外部入力と接続している。合成回路の外部出力数は常に1つであり、正常回路と故障回路の対応する外部出力対の排他的論理和をとったものの OR をその出力とする。明かに、テスト生成問題とは、この合成回路の出力を1にするような外部入力の組み合わせを求める問題となる。

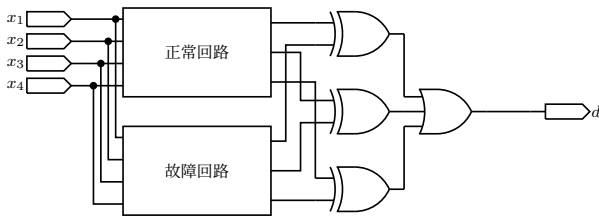


図 1 テスト生成用の合成回路

論理回路の構造から CNF 式を作る手法については多くの文献で紹介されているのでそれらを参照されたい [2]。

2.3 SAT ソルバを用いた手法の効率化

SAT ソルバを用いてテストパターンを求める手法の基本的

な概念は前節で説明した通りだが、実際にはいくつかの効率化手法が提案されている。

ひとつはテスト生成用の合成回路を作る際に正常回路と故障回路のうち、同一の構造となっている部分回路を共有するものである。具体的には故障回路の故障箇所よりも入力側にある部分回路がその共有すべき部分回路となる。つまり、故障回路として生成するのは故障箇所より出力側の部分回路のみとなる。以降、故障箇所より出力側の部分回路を「**故障コーン (fault cone)**」と呼ぶ。

ふたつめはファンアウトフリー領域の構造に着目した効率化手法である [5]。ただ一つのゲートのみに接続しているゲートを「**ファンアウトフリー (fanout free)**」と呼ぶ。連結しているファンアウトフリーなゲートの集合を「**ファンアウトフリー領域 (fanout free region)**」と呼ぶ。以降、本稿ではファンアウトフリー領域を FFR と略す。同じ FFR に属するゲートは出力側のゲートをたどることでもなく同一のファンアウトフリーでないゲートに到達する。このゲートを「**ファンアウトフリー領域の根**」と呼ぶ。FFR の定義から、任意のゲートからそのゲートの属している FFR の根のゲートまでの経路は常に一本であることがわかる。故障の伝搬経路が一本である場合は、単一経路活性化により検出条件が唯一に定まる。今、ゲート g がゲート r を根とする FFR に属しているとする。するとゲート g における値の反転が外部出力まで伝搬する条件 $P(g)$ はゲート g からゲート r まで伝搬する条件 $P^r(g)$ とゲート r から外部出力まで伝搬する条件 $P(r)$ を用いて以下の式 (1) で表される。

$$P(g) = P^r(g) \wedge P(r) \quad (1)$$

前述の単一経路活性化により $P^r(g)$ は関係するリテラルの論理積で表されるので SAT 問題を解く際には CNF 式として追加する必要がない。そこで、FFR の根のゲートの故障伝搬条件のみを求めておけばすべてのゲートの故障検出条件を求めることができる。

2.4 故障の支配関係と等価関係

故障 f_1 を検出するいかなるテストパターンも故障 f_2 を検出するとき、故障 f_1 は故障 f_2 を「**支配する**」と言う。もしくは故障 f_1 と故障 f_2 の間に「**支配関係**」があると言う。以降、故障 f_1 が故障 f_2 を支配している時、 $f_1 \succ f_2$ と書くこととする。2つの故障の間の支配関係においては、**反射律**と**推移律**が成り立つことは自明であるが、**対称律**や**反対称律**は成り立たない。

前述の故障の等価関係はこの支配関係を用いると次の様に定義できる。故障 f_1 と故障 f_2 が「**等価**」であるとは、 $f_1 \succ f_2$ かつ $f_2 \succ f_1$ が成り立つことである。この関係が、**反射律**、**対称律**、**推移律**を満たすことは自明である。よって、故障集合は等価関係によって定められる同値類に分割

することができる。今、元の故障集合を $\mathcal{F}$ として、 $\mathcal{F}$ の同値類から一つずつ故障を取り出して作る集合を $\mathcal{F}'$ とする。明らかに $\mathcal{F}'$ のすべての故障を検出するテストパターン集合は $\mathcal{F}$ の全ての故障を検出できる。この $\mathcal{F}'$ に対しては同じ同値類に属する故障は唯一つであるので、**反対称律**が成り立つ。すなわち、 $f_1 \succ f_2$ かつ $f_2 \succ f_1$ が成り立つのは $f_1 = f_2$ の時のみである。すると $\mathcal{F}'$ 上の支配関係は「**順序関係**」となる。

順序関係においては自身以外に先行者 (predecessor) を持たない極大元が存在する。この支配関係においては「自身以外に支配する故障の存在しない故障」のことである。この極大元以外の故障に関しては必ず自身を支配する故障が存在することになる。よって、 $\mathcal{F}'$ から極大元のみを取り出した集合 $\mathcal{F}''$ を考えた場合、 $\mathcal{F}''$ の全ての故障を検出するテストパターンは $\mathcal{F}'$ および $\mathcal{F}$ の全ての故障を検出するテストパターンとなることは自明である。よって、与えられた故障集合から他の故障に支配されない故障のみを取り出した集合を作ることによって考慮すべき集合の要素数を削減することができる。ただし、冗長故障は検出するテストパターンを持たないため、「冗長故障を検出するテストパターンが存在する」としたら、そのテストパターンは残りの全ての故障を検出する」という命題が成り立つ。そのため、冗長故障を含む故障集合においては冗長故障が唯一の極大元になってしまう*3。ここでは意味がないので対象の故障集合は冗長故障を含まないこととする。以降、このような故障集合 (支配関係における極大元の集合) を「**支配故障集合**」と呼ぶことにする。

2.5 故障の支配関係の検査

故障 f_1 が故障 f_2 を支配しているかどうかはテストパターン生成と同様に SAT ソルバを用いて調べることができる [4]。故障 f_1 を検出するすべてのテストパターンが故障 f_2 を検出するという事は、故障 f_1 を検出して故障 f_2 を検出しないテストパターンが存在しないということである。そこで、故障 f_1 を検出しつつ故障 f_2 を検出しない条件を作り SAT ソルバで解けばよい。この条件を表す回路イメージを図 2 に示す。

図 2 の回路に対して $d_1 = 1 \wedge d_2 = 0$ を満たすような入力 x_1, x_2, x_3, x_4 を求めることができたなら故障 f_1 は故障 f_2 を支配していない、ということになる。逆に条件を満たす入力が存在しない場合、故障 f_1 が故障 f_2 を支配していることになる。

図 2 の回路は図 1 の回路に比べるとサイズは大きくなるが、元のテストパターン生成問題が SAT ソルバで解ける場合には支配関係の検査も同程度かやや多めの計算時間で解くことができる。しかし、テストパターン生成問題の場合はその起動回数が高々故障数に比例するものであるのに対し

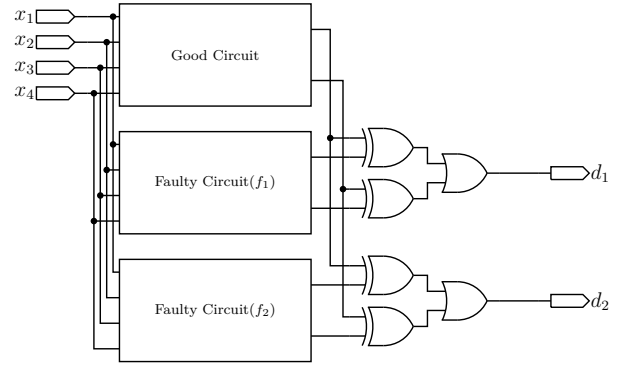


図 2 支配関係の検査用の合成回路

て、支配関係の検査の場合は最悪の場合、故障数の二乗に比例した回数の検査を行う必要がある。これでは例え一回の検査が高速に行えるとしても回路規模の増加に伴って実用的には計算不可能になってしまう。

2.6 同一の FFR 内の故障間の支配関係の検査

故障 f_1 が属している FFR の根のゲートを r とすると、故障 f の検出条件 $P(f_1)$ は $P^r(f_1) \wedge P(r)$ で表される。ここで、 $P^r(f_1)$ は故障 f_1 の影響が r まで伝搬する条件、 $P(r)$ は r における値の変化が外部出力まで伝搬する条件である。同じ FFR に属する別の故障 f_2 の検出条件も同様に $P^r(f_2) \wedge P(r)$ で表される。今、故障 f_1 が故障 f_2 を支配しているか検査することを考える。そのためには $C_{f_1 \succ f_2} = P(f_1) \wedge \neg P(f_2)$ が充足不可であることを証明する必要がある。 $P(f_1)$, $P(f_2)$ を展開すると $C_{f_1 \succ f_2}$ は以下の式で表される。

$$\begin{aligned} C_{f_1 \succ f_2} &= P(f_1) \wedge \neg P(f_2) \\ &= P^r(f_1) \wedge P(r) \wedge \neg(P^r(f_2) \wedge P(r)) \\ &= P^r(f_1) \wedge P(r) \wedge (\neg P^r(f_2) \vee \neg P(r)) \\ &= P^r(f_1) \wedge P(r) \wedge \neg P^r(f_2) \end{aligned}$$

つまり、故障 f_1 の影響が r まで伝搬し、 r における値の反転の影響が外部出力まで伝搬し、かつ、故障 f_2 の影響が r まで伝播しないという条件が充足不能の時、 $f_1 \succ f_2$ となる。これは故障 f_1 の検出条件 $P(f_1)$ に $\neg P^r(f_2)$ を加えたものである。前述のように FFR 内の故障伝播条件は非常に単純なのでこの条件の SAT 問題は故障 f_1 の検出条件とほぼ同程度の時間で解くことができる。

そこで、全故障対を対象に支配関係の検査を行う前に、同一の FFR に属する故障間の支配関係の検査を行って支配された故障の削減を行って対象故障数を減らしておくことは効果的である。支配関係による故障削減のうち、同一の FFR 内の故障対に関係する割合は後述の実験結果で明らかにする。

*3 複数の冗長故障が存在する場合でも定義によりそれらは同一の同値類に属しているので唯一と見なすことができる

2.7 故障シミュレーションを用いた検査対象候補対の削減

前述の様に故障 f_1 が故障 f_2 を支配していることを証明するためには図2の回路の対応したSAT問題が充足不能であることを示す必要がある。一方、支配していないことを示すには上述のSAT問題が充足可能であることを示せばよいので、もしも充足解がわかっていたらSATソルバを起動する必要はない。そこで、あるテストパターンを用いた故障シミュレーションを行った結果、故障 f_1 が検出され、かつ故障 f_2 が検出されなかった場合、SATソルバを用いなくても、故障 f_1 が故障 f_2 を支配していないことがわかる。

図3に故障シミュレーションを用いた単純な検査対象候補対の削減アルゴリズムを示す。

```

NAIVE.REDUCTION( $\mathcal{F}$ ) {
  FOR  $f$  IN  $\mathcal{F}$  {
    //  $f$  が支配する故障の候補集合を全故障集合で初期化する.
    cand_list( $f$ ) =  $\mathcal{F}$ 
  }
  DO {
    パタン  $p$  を生成する.
     $p$  を用いて故障シミュレーションを行う.
     $\mathcal{F}^+ = \{f | f \in \mathcal{F}, f \text{ は } p \text{ で検出された}\}$ 
     $\mathcal{F}^- = \mathcal{F} \setminus \mathcal{F}^+$ 
    FOR  $f$  IN  $\mathcal{F}^+ \{$ 
      cand_list( $f$ ) から  $\mathcal{F}^-$  に属している故障を取り除く
    }
  } WHILE(変化があった)
}
```

図3 NAIVE.REDUCTION アルゴリズム

前述の様にあるパタン p によって検出される故障 f_1 は検出されない故障 f_2 を支配することはない。そこで、パタン p によって検出された故障 f の支配する故障の候補集合 cand_list(f) から検出されなかった故障を取り除く。これを変化がなくなるまで繰り返せばSATソルバを用いることなく検査対象の候補対を削減することができる。このアルゴリズムの問題点は相変わらず対象故障数の二乗に比例した手間と記憶領域を必要とするということである。アルゴリズムの先頭で各故障の支配故障の候補を全故障集合で初期化しており、これだけで対象故障数の二乗に比例した手間と記憶領域を必要とすることがわかる。そのため故障数の増大に伴ってこのアルゴリズムを適用することは現実的に難しくなっている。

3. 二分法を用いた支配故障候補の削減手法

3.1 基本的なアイデアとアルゴリズム

故障シミュレーションの結果を用いて検査対象の故障対を削減すること自体は無駄なSATソルバの起動を抑えるために効果的であると考えられる。そこでより効率的な削

減手法を提案する。基本的なアイデアは故障を対象にするのではなく、故障グループ（故障集合）を対象にするというものである。初期状態は対象の故障を全て含んだ1つの故障集合とする。この故障集合に対してあるパターンを用いた故障シミュレーションを適用した結果で、検出された故障集合と検出されなかった故障集合に二分することができる*4この操作を複数のパターンを用いて繰り返すことでより詳細な細分化を行うことができる。故障数を N とした時、一回の二分操作にかかる計算の手間は $O(N)$ であり、理想の場合には $\log_2(N)$ 回の反復で十分な細分化を行えることが期待できる。等価故障はその性質上、いかなるパターンによっても区別されないため、常に同じ故障集合上に存在する。そのため、この二分法においては等価故障の多寡はほとんど全体の効率に影響しない。一方、単純な手法では常に等価故障の個数の二乗に比例した検査候補対を扱う必要があるため、大きな影響を受ける。

次に、この故障集合間の支配関係の候補の求め方について説明する。まず、この支配関係においては反射律が成り立つため、常に自身の故障集合は支配関係の候補となる。初期状態においてはただ一つの集合しか存在しないため、これが唯一の候補である。今、ある故障集合 $\mathcal{F}_1$ が支配関係の候補として $\mathcal{F}_2$ を持っているとする。これらの故障集合に対してパタン p を用いた故障シミュレーションを行って、それぞれ検出された故障集合 $\mathcal{F}_1^+$, $\mathcal{F}_2^+$ と検出されなかった故障集合 $\mathcal{F}_1^-$, $\mathcal{F}_2^-$ に細分化を行ったものとする。この時、細分化された故障集合間の支配関係の候補は以下の様になる。

- $\mathcal{F}_1^+ : \{\mathcal{F}_2^+\}$
- $\mathcal{F}_1^- : \{\mathcal{F}_2^+, \mathcal{F}_2^-\}$

このように検出されなかった故障集合 $\mathcal{F}_1^-$ に対しては支配関係の候補数が2倍に増える可能性がある。ただし、最悪の場合でも全故障数 N を超えることはない。

図4に提案手法である二分法を用いた支配故障候補の削減アルゴリズムを示す。

*4 場合によってはどちらか一方の集合が空集合になる可能性はある。

```

DICHOTOMY_REDUCTION( $\mathcal{F}$ ) {
  // 初期状態は全故障をひとかたまりにした故障集合のみとなる.
   $F_0 = \mathcal{F}$ 
  G.list =  $\{F_0\}$ 
  cand.list( $F_0$ ) =  $\{F_0\}$ 
  DO {
    パタン  $p$  を生成する.
     $p$  を用いて故障シミュレーションを行う.
    new_G.list =  $\phi$ 
    FOR  $F_i$  IN G.list {
       $F_i^+ = \{f | f \in F_i, f \text{ は } p \text{ で検出された}\}$ 
       $F_i^- = F_i \setminus F_i^+$ 
      new_G.list = new_G.list  $\cup \{F_i^+, F_i^-\}$ 
      FOR  $F_j$  IN cand.list( $F_i$ ) {
         $F_j^+ = \{f | f \in F_j, f \text{ は } p \text{ で検出された}\}$ 
         $F_j^- = F_j \setminus F_j^+$ 
        cand.list( $F_i^+$ ) に  $F_j^+$  を追加する.
        cand.list( $F_i^-$ ) に  $F_j^+, F_j^-$  を追加する.
      }
    }
    G.list = new_G.list
  } WHILE(変化があった)
}

```

図 4 DICHOTOMY_REDUCTION アルゴリズム

3.2 複数パタンを用いた多分法の拡張

一般に故障シミュレーションではワードレベルのビット並列性を活かすために複数のパタンを 1 ワードに圧縮してパタン並列シミュレーションを行うことが可能となっている。今、 k 個のパタンを用いて故障シミュレーションを行ったとすると結果は個々のパタンに対して検出できたかできなかったかを表す k ビットのビットベクタとなる。よって故障集合 F は最大で 2^k 個の故障集合に細分化されることになる。個々の故障集合は結果を表す k ビットのビットベクタと関連付けられる。細分化された故障集合の支配関係の候補はこのビットベクタに従って以下の様に定義される。

- k ビットのビットベクタ b_1 と b_2 に対して、 b_1 の 1 になっているビット位置に対しては b_2 のビットも 1 になっている時、 b_1 は b_2 を**支配**すると定義する。

このビットベクタの支配関係の検査はビットワイズ論理演算命令で高速に実行できるので k 回二分法を適用するよりも高速な処理が可能となる。

3.3 等価故障の検査

DICHOTOMY_REDUCTION アルゴリズムの最終結果は故障集合のリスト G.list と各々の故障集合 F に対する支配故障の検査対象のリスト cand.list(F) である。単純にはこれらの情報を故障集合の要素である故障に反映させればよい。多くの場合、細分化された故障集合の要素数は 1 になっており、対応付けも容易である。一方、複数の故障を

要素とする故障集合があった場合、それらの故障は等価故障である可能性が高い。そこで、そのような故障集合を対象に等価故障の検査を行う。等価故障の検査が成功した場合、どれか一つの故障を代表故障として残し、残りの故障を削除する。等価故障の検査が失敗した場合、結果として反例となるパタンが一つ求められているのでそのパタンを用いて再度 DICHOTOMY_REDUCTION アルゴリズムを適用する。結果として最低でも 1 つ以上の故障集合が分割されることになる。以上の処理を変化がなくなるまで繰り返す。

3.4 支配故障の検査

等価故障の検査が終了したときには各故障集合の要素数は 1 になっているはずである。そこで、故障集合に対して定義された支配関係を故障に対する支配関係に置き換える。この操作は一对一の対応になっているので簡単に行うことができる。その後、得られた支配故障の候補対に対して SAT ソルバを起動して支配関係の検査を行えばよい。注意が必要なのは支配関係の検査が失敗した場合で、この場合は等価故障の検査と同様に得られたパタンを用いて再度 DICHOTOMY_REDUCTION を行うことで支配関係候補の更新を行う。

4. 実験結果

以上のような支配関係を利用して故障数の削減を行うプログラムを開発し、ベンチマーク回路を用いた実験を行った。実験の概要は以下の通りである。

- ISCAS85, ISCAS89, ITC99 のベンチマーク回路のうち、故障数が 1000 以上の回路を対象とする。順序回路の場合、スキャン方式で FF の入力と出力をそれぞれ疑似出力と疑似入力とみなした組み合わせ回路として扱う。
- 全故障に対して通常のテストパタン生成を行い、検出可能故障と冗長故障の特定を行う。検出可能故障に対しては対応するテストパタンを記録しておく。
- FFR 単位で支配故障の検査を行い、対象故障の削減を行う (2.6 節)。
- DICHOTOMY_REDUCTION もしくは NAIVE_REDUCTION を行い、支配故障の検査対象の候補対を求める。その際に、最初は各故障を検出するテストパタンを用い、残りはランダムパタンを用いる。
- 等価故障の検査 (3.3) と支配故障の検査 (3.4) を行い、支配故障を求める。

使用計算機は Intel Core i7-14700 (2.112GHz, 28 コア, メモリ 32GB), OS は FreeBSD 15.0-RELEASE-p5, 使用コンパイラは clang++-19.1.7 である。プログラムはマルチスレッドであり、複数のコアを用い

ている。ただし、DICHOTOMY_REDUCTION および NAIVE_REDUCTION はそのアルゴリズムの性質上、並列化が難しいのでシングルスレッドで動かしている。また、SAT ソルバは MINISAT と同様のアルゴリズムを用いた自作のプログラムを用いている。

表 1 に支配故障による故障数の削減効果を示す。なお、回路名は初期故障数の昇順に並べ替えている。

表 1 故障の削減数

回路名	初期故障数	FFR 単位の削減	最終結果
C1355	1574	982(62.39)	462(29.35)
C1908	1879	1334(71.00)	525(27.94)
C2670	2747	1863(67.82)	962(35.02)
C3540	3428	2405(70.16)	1239(36.14)
C5315	5350	3511(65.63)	1781(33.29)
C7552	7550	4898(64.87)	1678(22.23)
C6288	7744	4862(62.78)	2202(28.43)
s13207	9815	6744(68.71)	2792(28.45)
s15850	11725	7372(62.87)	3230(27.55)
b15	21988	15043(68.41)	8798(40.01)
b14	22802	14606(64.06)	7991(35.05)
s38417	31180	21403(68.64)	10813(34.68)
s38584	36303	22451(61.84)	11836(32.60)
s35932	39094	22239(56.89)	10252(26.22)
b20	45459	28738(63.22)	15873(34.92)
b21	46154	29224(63.32)	16512(35.78)
b22	67536	42845(63.44)	23877(35.35)
b17	76625	51002(66.56)	28395(37.06)

FFR 単位の処理で概ね 30%~40%の故障を削減している。その後全体的な処理で残りの 30%~40%の故障を削減している。ほとんどの回路で 30%程度まで故障数を削減しており支配故障を用いた削減が効果的であることを示している。

次に表 2 に DICHOTOMY_REDUCTION と NAIVE_REDUCTION を用いた削減処理の全体の時間 (単位は秒) を示す。

表 2 処理時間の比較

回路名	DICHOTOMY_REDUCTION	NAIVE_REDUCTION
C1355	0.93 (0.14)	1.01 (0.25)
C1908	2.01 (0.90)	1.87 (0.73)
C2670	3.11 (0.71)	3.74 (1.24)
C3540	6.49 (3.27)	6.48 (3.32)
C5315	4.92 (3.30)	7.57 (6.01)
C7552	11.93 (5.92)	14.47 (8.06)
C6288	227.69 (0.80)	231.18 (1.87)
s13207	20.50 (9.97)	55.45 (42.71)
s15850	40.95 (14.65)	75.02 (46.66)
b15	283.49 (62.43)	530.33 (311.14)
b14	1130.26 (62.12)	1320.71 (269.93)
s38417	340.36 (161.88)	1366.55 (1174.01)
s38584	272.29 (129.04)	1138.97 (1015.75)
s35932	1054.56 (27.49)	1615.66 (689.78)
b20	2453.44 (218.86)	3987.11 (1769.90)
b21	2505.10 (288.58)	4310.48 (2044.15)
b22	3978.21 (596.23)	9438.67 (6086.93)
b17	2508.35 (1175.00)	14563.80 (13228.13)

各項目のカッコのなかは DICHOTOMY_REDUCTION および NAIVE_REDUCTION に要した時間である。提案手法である DICHOTOMY_REDUCTION の場合、ITC99 の大規模なベンチマーク回路に対しても比較的短時間ですべての処理を行っている。一方、NAIVE_REDUCTION の場合は大規模なベンチマーク回路の場合には処理時間の大部分を占めている。例えば b17 の場合は等価故障と支配故障の検査に要した時間は約 1300 秒であるのに対して NAIVE_REDUCTION に約 10 倍の 13000 秒を要している。これでは支配故障の検査時間を短縮するための前処理として効率的な処理とは言えない。なお、どちらの手法を用いても等価故障と支配故障の検査に要した時間は変わらない。

次に故障シミュレーションを用いた検査対象故障の絞り込み効果について考える。表 3 に DICHOTOMY_REDUCTION 後の検査対象故障対の数を示す。NAIVE_REDUCTION でも同一の結果となる。

総数は故障数に応じて増加しているが平均は概ね 10 以下の値となっている。s35932 だけは約 100 個の多めの値となっている。s35932 は DICHOTOMY_REDUCTION が比較的短い時間で終了しており、少ないパターンで変化がなくなったため多くの候補が残ってしまった可能性がある。故障シミュレーションで用いるパターンに関しては検討の必要があると考えられる。いずれにせよ、もともとは全故障数の二乗に比例した故障対が存在しているのに対して故障シミュレーションを用いた絞り込みを行うことでほぼ故障数に比例したオーダーまで候補数を削減することができるといえる。ただし、検査した故障対のうち、実際に支配関係が証明できた割合 (成功率) に関しては、大規模回路に対しては概ね低い値となっている。とはいえ、この処理に

回路名	故障対の総数	1 故障当たりの平均数	検査が成功した数	成功率
C1355	525	0.53	520	0.99
C1908	926	0.69	809	0.87
C2670	9729	5.22	901	0.09
C3540	1590	0.66	1166	0.73
C5315	1763	0.50	1730	0.98
C7552	6295	1.29	3216	0.51
C6288	2660	0.55	2660	1.00
s13207	22822	3.38	3927	0.17
s15850	44444	6.03	4117	0.09
b15	46344	3.08	6094	0.13
b14	45153	3.09	6586	0.15
s38417	344405	16.09	10541	0.03
s38584	126388	5.63	10587	0.08
s35932	2216204	99.65	11983	0.01
b20	72180	2.51	12755	0.18
b21	75227	2.57	12597	0.17
b22	119532	2.79	18841	0.16
b17	577344	11.32	22159	0.04

表 3 検査対象故障対数

時間をかけると全体の処理時間の増大を招くため、効率的な処理を行うためにどのような方法が適しているのかを検討する必要がある。

5. おわりに

本稿では故障の支配関係を用いてテストパターン生成問題で考慮すべき対象の故障数を大幅に減らせることを示した。ベンチマーク回路を用いた実験では考慮すべき故障数もとの故障数の約30%~40%に削減できている。このアイデアそのものは昔から知られていたが、大規模回路に対して効率よく支配故障の検査を行う手法は提案されていなかった。本稿では二分法を用いて効率よく支配故障の検査対象候補を列挙するアルゴリズムを提案した。ベンチマーク回路を用いた実験では提案手法である DICHOTOMY_REDUCTION は大規模回路に対しても効率よく検査対象の故障対を求めることが示されている。今後は候補の絞り込みを効果的に行えるパタンの求め方などの検討を行う予定である。

参考文献

- [1] Davis, M. and Putnam, H.: A Computing Procedure for Quantification Theory, Journal of the ACM, Vol. 7, pp. 201–215 (1960).
- [2] Larrabee, T.: Test pattern generation using Boolean satisfiability, IEEE Transactions on Computer-Aided Design of Integrated Circuits, Vol. 11, No. 1, pp. 4–15 (1992).
- [3] Marques-Sliva, J. and Sakallah, K.: GRASP: A Search Algorithm for Propositional Satisfiability, IEEE Transactions on Computer-Aided Design of Integrated Circuits, Vol. 48, No. 5, pp. 506–521 (1999).
- [4] Matsunaga, Y.: A Test Pattern Compaction Method Using SAT-Based Fault Grouping, IEICE TRANSACTIONS on Fundamentals, Vol. E99-A, No. 12, pp.

2302–2309 (online), DOI: 10.1587/transfun.E99.A.2302 (2016).

- [5] Matsunaga, Y.: An Accelerating Technique for SAT-based ATPG, IPSJ Transactions on System and LSI Design Methodology, Vol. 10, pp. 39–44 (online), DOI: 10.2197/ipsjtsldm.10.39 (2017).
- [6] Moskewicz, M., Madigan, C., Zhao, Y., Zhang, L. and Malik, S.: Chaff: Engineering an Efficient SAT Solver, Proceedings of the 38th Design Automation Conference (2001).
- [7] Zhang, L. and Malik, S.: Efficient Conflict Driven Learning in a Boolean Satisfiability Solver, Proceedings of IEEE International Conference on Computer Aided Design (ICCAD 2001) (2001).